IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, VOL. 33, NO. 1, JANUARY 2022

멀티테넌트 GPU 클러스터에서의 탄력적 딥러닝

Yidi Wu, Kaihao Ma, Xiao Yan, Zhi Liu, Zhenkun Cai, Yuzhen Huang, James Cheng, Han Yuan, Fan Yu

초록

본 연구는 GPU 클러스터에서 심층 신경망(DNN) 훈련을 위한 탄력성(elasticity), 즉 병렬도(GPU 수)를 동적으로 조정하는 능력을 지원하는 방법을 연구한다. 탄력성은 클러스터 부하 변동에 따른 다양한 스케줄링 목표(예: 작업 처리량, 작업 완료 시간, GPU 효율성) 달성, 유휴 자원의 일시적 활용, 성능 프로파일링·작업 마이그레이션·낙오자(straggler) 완화 지원 등 여러 측면에서 멀티테넌트 GPU 클러스터 관리에 도움이 된다. 우리는 간단한 API로 탄력적 딥러닝을 가능하게 하며 TensorFlow 및 PyTorch와 같은 기존 딥러닝 프레임워크와 쉽게 통합할 수 있는 EDL을 제안한다. EDL은 무중단 스케일링(stop-free scaling) 및 동적 데이터 파이프라인과 같이 병렬도 조정의 오버헤드를 줄이는 데 필요한 기술도 포함한다. 실험을 통해 EDL이 GPU 클러스터 관리의 위 응용 분야에서 실제로 상당한 이점을 가져다줌을 보인다.

색인어: 딥러닝 시스템, 탄력적 딥러닝, GPU 클러스터 관리

1. 서론

딥러닝(DL)의 큰 성공으로 인해 많은 조직이 심층 신경망(DNN) 훈련을 위한 대규모 GPU 클러스터를 구축하고 있다. GPU 클러스터는 일반적으로 다수의 동시 사용자에게 서비스를 제공한다. 사용자는 DNN 훈련 작업과 해당 자원 요구사항(예: GPU 수)을 클러스터에 제출한다. 멀티테넌트 GPU 클러스터는 보통 전통적인 클러스터 관리자(예: YARN, Mesos)나 GPU 클러스터에 특화된 스케줄러(예: Optimus, Gandiva, Tiresias, Themis, SLAQ)에 의해 관리되며, 높은 처리량, 높은 GPU 효율성, 짧은 작업 완료 시간(JCT), 소규모 작업에 대한 양호한 응답성 등의 목표를 추구한다.

Microsoft의 프로덕션 GPU 클러스터 트레이스 데이터 분석을 통해, 우리는 탄력성, 즉 DNN 훈련 작업의 병렬도(GPU 수)를 조정하는 능력이 여러 측면에서 멀티테넌트 GPU 클러스터 관리에 유익함을 발견했다.

- **클러스터 부하 변동에 적응.** GPU 클러스터는 오랜 기간 과부하 상태이다가 다른 기간에는 활용도가 낮을 수 있다. 탄력성을 통해 DNN 훈련 작업을 스케일 인/아웃(즉, 병렬도 증감)할 수 있어, 클러스터가 바쁘지 않거나 유휴 자원이 일시적으로 있을 때 높은 처리량을 달성하고, 클러스터가 과부하일 때 GPU 효율성을 향상시킬 수 있다.
- **우선순위 기반 스케줄링 적용.** 탄력성을 사용하여 우아하게 우선순위를 적용할 수 있다. 저우선순위 작업을 선점하는 대신, 저우선순위 작업을 스케일 다운하여 고우선순위 작업에 충분한 자원을

확보할 수 있다. 한 가지 활용 시나리오는 소규모 작업을 우선순위화하여 선두 차단(head-of-line blocking)을 완화하고 작업 응답성과 평균 JCT를 개선하는 것이다.

- **유연한 작업 관리 지원.** 탄력성은 낙오자 완화, 성능 프로파일링, 워커 마이그레이션과 같은 클러스터 관리 기능에 유용하다: 1) 낙오자는 동기식 훈련의 처리량에 해가 되며, 작업을 스케일 인하여 제거할 수 있다. 2) 서로 다른 병렬도에서의 작업 성능은 일련의 스케일 인 연산을 통해 쉽게 프로파일링할 수 있다. 3) 스케일 인과 스케일 아웃을 사용하여 작업의 워커를 한 머신에서 다른 머신으로 마이그레이션할 수 있어 자원 파편화를 줄일 수 있다.

탄력성은 그래프 처리, 배치 처리, 파라미터 서버 기반 머신러닝 등 다른 유형의 워크로드에도 탐구되어 왔다. 그러나 DL 훈련 작업과 멀티테넌트 GPU 클러스터의 고유한 특성은 앞서 언급한 탄력성의 이점을 달성하는 데 어려움을 제기한다. 구체적으로, 탄력성의 오버헤드가 낮아야 잦은 병렬도 조정(예: 스케줄링)과 일시적 유휴 자원의 효과적 활용을 지원할 수 있다. 병렬도는 Stop-Resume으로 사소하게 조정할 수 있는데, 이는 작업을 체크포인트로 저장하고 원하는 병렬도로 재시작하는 방식이다. 하지만 실행 중인 작업은 보통 30초 이상 멈춰야 한다. 또한, 탄력성은 기존 DL 시스템과 호환 가능하고 최소한의 사용자 노력을 요구해야 실제 채택이 가능하다. 마지막으로, 일관성과 효율성 사이의 균형이 유지되어야 한다.

우리는 멀티테넌트 GPU 클러스터에서 탄력적 딥러닝을 지원하기 위해 **EDL**을 제안한다. EDL은 클러스터 스케줄러와 DL 시스템 사이의 경량 조정 계층이다. EDL은 단일 머신 실행을 기반 DL 시스템(예: TensorFlow, PyTorch, MindSpore, MXNet)에 위임한다. DL 시스템은 EDL로부터 훈련 데이터 블록의 메타데이터를 가져오고 미니배치 완료 후 EDL에 알리기만 하면 된다. EDL은 다양한 DL 시스템에 플러그인으로 사용할 수 있으며, 사용자는 원래 스크립트에 몇 줄만 추가하면 탄력성을 누릴 수 있다. 스케줄러는 EDL에 간단한 API(예: `scale_in()`, `scale_out()`)를 사용하여 훈련 작업에 대해 워커를 추가/제거하도록 지시할 수 있다. 가장 중요한 것은, EDL이 Stop-Resume에 비해 탄력성의 오버헤드를 크게 줄인다는 점이다.

EDL은 어떤 워커든 언제든지 작업에 참여/이탈할 때 정확성과 효율성을 보장하도록 설계되었다. EDL에서 각 작업은 리더 프로세스에 의해 관리되며, EDL은 빠른 리더 선출을 위해 분산 트랜잭션 기반 메커니즘을 사용한다. 스케일 아웃을 위해 EDL은 무중단 스케일링(stop-free scaling)을 제안하는데, 이는 새로 추가된 워커가 실행 준비되는 동안 기존 워커가 훈련을 계속할 수 있게 한다. 스케일 인을 위해 EDL은 미니배치 끝에서 워커를 제거하는 우아한 종료(graceful exit)를 사용하여 무시할 수 있는 오버헤드를 달성한다. 데이터 준비를 위해 EDL은 동적 데이터 파이프라인을 사용하여 온디맨드 방식으로 워커에게 데이터 블록을 할당하고, GPU 기아(starvation)를 피하기 위해 데이터 프리페칭을 활용한다.

EDL의 성능을 검증하기 위해 광범위한 실험을 수행했다. 먼저 EDL이 정상 훈련(탄력적 스케일링 없이)에서 Horovod에 비해 낮은 오버헤드를 가짐을 보인다. 탄력적 스케일링에 대해서는, EDL이 스케일 아웃의 오버헤드를 한 자릿수(order of magnitude) 수준으로 줄이며, Stop-Resume에 비해 스케일 인의 오버헤드가 무시할 수 있음을 보인다. 몇 가지 간단한 수정으로 최신 DL 작업 스케줄러인 Tiresias에서 탄력성을 효율적으로 적용하여, 평균 JCT를 89.5%까지 줄일 수 있었다.

2. 동기

2.1 배경

DNN 모델은 데이터셋을 여러 번 반복(에포크라 함)하여 훈련되며, 각 에포크에서 데이터셋은 무작위로 섞이고 여러 미니배치로 분할된다. 각 미니배치마다 모델은 확률적 경사 하강법(SGD) 또는 Adam, AdaGrad 같은 변형으로 업데이트된다. 그래디언트 계산은 행렬 곱셈과 같은 계산 집약적 커널을 포함하므로, DNN 훈련은 보통 GPU에서 수행된다.

데이터의 증가하는 양과 DNN 모델(예: ResNet, VGG, Inception)의 높은 복잡성으로 인해, DNN 훈련은 보통 단일 GPU에서 합리적인 시간 내에 완료될 수 없으므로, 다중 GPU에서의 분산 훈련이 좋은 대안이 된다. 다양한 분산 훈련 방식 중에서 동기식 데이터 병렬(synchronous data-parallel)이 가장 널리 사용되며, 이는 데이터셋을 GPU 간에 분할하고 각 GPU(즉, 워커)가 일부 훈련 샘플에 대한 그래디언트를 병렬로 계산한다. 모든 워커가 미니배치의 그래디언트 계산을 마치면, 워커들의 로컬 그래디언트가 집계되어 모델에 반영된다. 매 미니배치 끝에 동기화 배리어가 적용되므로, 낙오자(straggler)는 동기식 훈련의 성능에 해가 된다.

Allreduce는 분산 워커의 모델 업데이트를 조정하는 인기 있는 프로토콜로, TensorFlow, PyTorch, MXNet 및 Horovod에 널리 채택되어 있다. Ring-Allreduce 구현에서 워커들은 링 통신 토폴로지를 형성하고, 각 워커는 링의 인접한 두 이웃과만 통신한다. 그래디언트 텐서가 준비되면, 각 워커는 라운드 로빈 방식으로 인접 워커에게 텐서의 1/N(N은 워커 수)을 보내고, 받고, 집계한다. N-1 단계 후 각 워커는 모든 워커의 업데이트를 집계한 텐서의 1/N을 가진다. **파라미터 서버**도 분산 머신러닝에 널리 사용되며, 모델 업데이트/조회를 위한 키-값 인터페이스를 제공한다.

Horovod는 Allreduce 기반 분산 DNN 훈련의 최신 프레임워크이다. Horovod는 탄력적 훈련에 대한 네이티브 지원이 없다. 병렬도를 동적으로 조정하기 위해 Horovod는 기반 DL 시스템이 제공하는 Stop-Resume 메커니즘을 사용할 수 있다. Horovod 작업에서 리더 프로세스가 워커 간의 그래디언트 동기화 순서와 세분도를 조정한다.

2.2 탄력성의 이점

우리는 실험 관측과 Microsoft의 트레이스 데이터를 통해 멀티테넌트 GPU 클러스터 관리에 대한 탄력성의 이점을 보인다. 트레이스 데이터는 스케줄링 이벤트와 Microsoft의 프로덕션 GPU 클러스터(약 2,300개 GPU)에서 약 2개월간 수집된 작업에 대한 간략한 설명을 포함한다.

처리량과 GPU 효율성 조정. 두 가지 스케일링 방식을 고려한다. 약한 스케일링(weak scaling)은 GPU당 배치 크기를 고정하여, 합산 배치 크기가 병렬도에 따라 증가한다. 약한 스케일링에서 처리량이 병렬도에 거의 선형적으로 증가하고 GPU 효율성이 거의 일정하게 유지된다. 강한 스케일링(strong scaling)은 합산 배치 크기를 고정하고 병렬도에 따라 GPU당 배치 크기를 조정한다. 강한 스케일링은 알고리즘 설계에 대해 탄력성을 투명하게 만들지만, GPU 효율성이 병렬도 증가에 따라 선형적으로 감소한다. 이는 통신 비용이 병렬도에 따라 증가하고 GPU당 배치 크기가 작을 때 통신과 계산을 겹치기 어렵기 때문이다.

따라서 탄력성을 사용하여 다양한 스케줄링 목표(예: 더 짧은 JCT, 높은 처리량 또는 GPU 효율성)에 따라 DNN 훈련 작업의 병렬도를 동적으로 조정할 수 있다.

클러스터 활용률과 JCT 개선. 클러스터는 일부 기간에 거의 완전히 부하가 걸리고 많은 작업이 대기 중인 반면, 다른 기간에는 부하가 비교적 낮다. GPU의 유휴 간격(idle interval)을 이전 작업 완료와 다음 작업 시작 사이에 경과한 시간으로 정의한다. 유휴 간격은 멱함수 분포를 따르며 대부분이 짧은 간격이다. 구체적으로, 유휴 간격의 39.62%가 4분 미만이며, 이는 피크 시간 동안 유휴 자원의 41.5%를 차지한다. 탄력성을 사용하여 작업을 스케일 인/아웃하여 클러스터 부하 변동에 적응하고 일시적 유휴 자원을 활용할 수 있다.

응답성 개선. 트레이스 데이터에서 작업 크기에 큰 변동이 있다. 소규모 작업의 경우 약 50%가 (대기시간/실행시간) 비율이 1보다 크며, 이는 이 소규모 작업들의 대기 시간이 실행 시간보다 길다는 것을 의미한다. 실제로 소규모 작업의 약 20%는 실행 시간의 10배에 달하는 대기 시간을 가진다. 이 문제를 해결하기 위해, 클러스터가 과부하일 때 일부 장기 실행 작업을 스케일 인하여 소규모 작업을 위한 공간을 확보할 수 있다.

2.3 설계 원칙

섹션 2.2의 관찰을 바탕으로, 탄력성의 이점을 누리기 위해 다음과 같은 시스템 설계 원칙을 도출한다.

첫째, **탄력성은 낮은 오버헤드를 수반해야 한다.** 탄력성은 Stop-Resume을 통해 사소하게 달성할 수 있는데, 이는 작업을 체크포인트로 저장하고 원하는 병렬도로 재시작하는 방식이다. TensorFlow에서 1-GPU 작업을 다른 병렬도로 조정하는 비용은 40초에서 60초 이상이다. 이 높은 오버헤드는 동적 자원 가용성과 작업 요구사항에 적응하는 능력을 제한하며, 일시적 유휴 자원의 효과적 활용도 방해한다. 따라서 병렬도 조정의 오버헤드를 세심하게 숨기는 일련의 절차를 설계해야 한다.

둘째, **호환성과 사용성이 중요하다.** TensorFlow와 PyTorch와 같은 DL 시스템은 널리 사용되므로, 탄력성 제공은 새로운 시스템을 처음부터 개발하는 것이 아니라 이러한 노력 위에 구축되어야 한다. 사용자 관점에서는 기존 DL 시스템에서 작성된 스크립트를 탄력적 훈련 작업으로 실행하는 데 거의 노력이 필요하지 않아야 한다. 또한 스케줄러가 클러스터에서 작업의 병렬도를 조정하는 것이 간단해야 한다.

셋째, **투명성과 일관성이 고려되어야 한다.** 탄력성을 적용하는 작업의 실행이 고정 병렬도에서의 실행과 동등함을 보장해야 한다. 이를 위해 DNN 훈련 작업에 대해 적절한 일관성 의미론을 큰 오버헤드 없이 제공해야 한다.

3. 시스템 아키텍처와 API

우리는 분산 DL의 지배적 패러다임인 데이터 병렬, 동기식 훈련에 초점을 맞춘다.

DL 사용자들은 TensorFlow, PyTorch, MXNet과 같은 인기 시스템에 익숙하므로, 탄력성의 핵심 로직을 다양한 DL 시스템 간에 공유할 수 있다면 바람직할 것이다. 따라서 우리는 EDL을 DL 시스템과 GPU 클러스터 관리자 사이에 위치하는 조정 계층으로 설계한다. EDL의 주요 API는 표 1에 요약되어 있다.

표 1. EDL의 주요 API

클러스터 스케줄러용 API	설명
`scale_in(job_handle, rmv_GPU_info)`	작업에서 GPU 제거
`scale_out(job_handle, add_GPU_info)`	작업에 GPU 추가
`profile(job_handle, min_p, max_p)`	작업 프로파일링
DL 시스템용 API	**설명**
`elastic_shard_generator()`	다음 샤드의 정보 생성
`notify_batch_end()`	스케일링 필요 여부 확인

클러스터 관리자는 EDL의 스케줄러 API를 사용하여 병렬도 조정 절차의 세부사항을 알 필요 없이 작업의 병렬도를 조정할 수 있다. 사용자는 기존 DL 시스템을 사용하여 훈련 스크립트를 작성하고 EDL을 사용하기 위해 몇 줄만 추가하면 된다.

3.1 클러스터 스케줄러용 API

중앙 집중식 클러스터 스케줄러(예: YARN)가 자원 가용성과 작업 상태를 파악하여 스케줄링 결정을 내린다고 가정한다. 스케줄러는 고유한 `job_handle`로 식별되는 작업의 병렬도를 `scale_in()`과 `scale_out()` 연산자를 사용하여 EDL에 지시할 수 있다. 스케일링 연산자가 호출되면, 작업을 실행하는 워커들의 리더에게 메시지가 전송된다. 리더는 지정된 GPU의 제거/추가를 조정하고, 조정 완료 후 스케줄러에 확인 메시지를 보낸다.

`profile()` 연산자는 [min_p, max_p]로 지정된 범위의 병렬도에서 작업의 처리량과 GPU 효율성을 측정한다. 이는 작업의 최적 병렬도를 찾거나 스케줄링을 위한 정보를 수집하는 데 사용할 수 있다.

EDL은 스케줄러의 개입 없이 장애로부터 자동으로 작업을 복구하여, 재스케줄링 및 재시작으로 인한 지연을 제거한다. EDL은 약한 스케일링과 강한 스케일링을 모두 지원하지만, 기본적으로 강한 스케일링을 사용한다. 또한 EDL은 에포크 내에서 데이터셋을 반복 없이, 누락 없이 한 번 처리하도록 보장한다.

3.2 DL 시스템용 API

EDL은 인기 있는 DL 시스템 사용자가 자신의 스크립트를 탄력적 작업으로 실행할 수 있도록 간단한 API를 제공한다. 일부는 Horovod의 것과 표준적이고 유사하며(예: `init()`, `shutdown()`, `all_reduce()`), `elastic_shard_generator()`와 `notify_batch_end()`는 탄력성을 위해 특별히 도입되었다.

`elastic_shard_generator()`는 제너레이터 객체를 반환하며, `next()` 메서드 호출 시 워커에 훈련 샘플 청크의 메타데이터를 제공한다. DL 시스템은 이를 사용하여 파티션 목록에서 동적으로 훈련 샘플을 로드할 수 있다. 이 연산자는 스케일링 하에서 동적인 워커 집합에 훈련 샘플의 효율적 분배를 보장한다. EDL은 미니배치 끝에서 작업에 워커를 추가/제거하므로 훈련 진행이 손실되지 않으며, 사용자는 `notify_batch_end()`를 호출하여 EDL에 미니배치 경계를 알릴 수 있다.

TensorFlow에서 EDL을 사용하는 예제 코드(Listing 1)에서, 3행은 EDL 데몬을 초기화하고, 5-6행은 EDL의 `elastic_shard_generator()` 메서드로부터 TensorFlow 데이터셋 객체를 생성한다. 9행의 `edl.Optimizer`는 TensorFlow의 옵티마이저 클래스를 상속하는 헬퍼 클래스이며, Allreduce 연산을 계산 그래프에 삽입한다. 16행에서 사용자는 `notify_batch_end()`로 미니배치의 끝을 표시한다. EDL 사용이 간편하며 사용자 스크립트에 몇 줄(즉, "edl"이 포함된 행들)만 추가하면 됨을 알 수 있다.

4. 시스템 설계 및 구현

EDL 시스템의 설계는 세 가지 목표를 가진다: **유연성**, **효율성**, **일관성**. 유연성은 어떤 프로세스(워커든 리더든)든 언제든지 작업을 떠나거나 참여할 수 있어, 스케줄러가 유연하게 병렬도를 조정할 수 있음을 의미한다. 효율성은 EDL이 Stop-Resume에 비해 병렬도 조정 오버헤드를 크게 줄이고, 정적 병렬도 훈련에 무시할 수 있는 오버헤드를 도입해야 함을 의미한다. 일관성은 스케일링 하에서의 작업 실행이 스케일링 없는 실행과 동등해야 함을 의미한다.

이 목표를 달성하기 위해 EDL은 세 가지 핵심 설계를 채택한다: **자동 작업 관리**(섹션 4.1), **효율적 병렬도 조정**(섹션 4.2), **동적 데이터 파이프라인**(섹션 4.3).

4.1 자동 작업 관리

각 작업은 워커를 관리하는 리더를 가진다. 그러나 스케일링이나 장애로 인해 리더가 작업을 떠날 수 있다. 각 워커는 리더가 알려지지 않을 때마다 리더 선출/발견 절차를 수행하여, 항상 작업을 관리하는 리더가 존재하도록 보장한다.

구체적으로, 작업이 시작되면 각 워커는 먼저 리더 선출 절차를 수행하며, 이는 ZooKeeper나 etcd와 같은 외부 조정 시스템을 사용한 분산 compare_and_swap 트랜잭션으로 구현된다. 측정에 따르면, 리더 선출은 평균 7ms, 256개 워커가 분산 조정을 위해 etcd를 사용할 때 최대 33ms가 소요되었다.

리더가 선출되면, RPC 서버를 설정하여 연결을 수락하고, 다른 워커는 리더에 연결하여 작업 참여를 위한 등록 메시지를 보낸다. 작업 실행 중에 리더는 매 미니배치의 그래디언트 동기화 요청으로부터 워커의 활성 상태를 추론하므로, 명시적 하트비트 메시지가 불필요하다.

4.2 효율적 병렬도 조정

병렬도 조정의 오버헤드를 줄이기 위해, EDL은 무중단 스케일링(stop-free scaling)을 사용하여 `scale_out()` 중에 실행 컨텍스트 준비의 높은 비용을 숨기고, 우아한 종료(graceful exit)를 적용하여 `scale_in()` 의 오버헤드를 무시할 수 있게 만든다.

스케일 아웃. 실행 중인 작업에 새 워커를 추가하는 것은 세 단계를 포함한다: 실행 컨텍스트 준비, 통신 토폴로지 구성, 모델 준비. 실행 컨텍스트 준비는 동적 라이브러리(예: cuDNN, cuBLAS) 로딩, 훈련 데이터 준비, GPU 및 주 메모리 공간 할당 등을 포함한다.

우리는 **무중단 스케일링**을 제안한다. 핵심 통찰은 기존 워커의 훈련이 새 워커의 실행 컨텍스트 준비 중에 멈출 필요가 없다는 것이다. 각 새 워커는 두 개의 별도 스레드(메인 스레드와 백그라운드 스레드)를 실행한다. 메인 스레드는 실행 컨텍스트 준비를 수행하고, 동시에 백그라운드 스레드는 리더에게 등록 요청을 보낸다. 리더는 새 워커를 포함하는 새 통신 토폴로지를 구성하고 모든 워커에게 전달한다. 원래 통신 토폴로지는 아직 해제되지 않았으므로, 기존 워커는 영향을 받지 않고 훈련을 계속할 수 있다.

모든 새 워커로부터 준비 메시지를 받으면, 리더는 OK 메시지와 미래 타임스탬프를 모든 워커에게 전달한다. 기존 워커는 `notifyBatchEnd()`에 의해 표시되는 매 미니배치 끝에서 확인하고, 로컬 타임스탬프가 리더가 지정한 타임스탬프에 도달하면 새 통신 토폴로지로 전환한다.

스케일 인. `scale_in()`에는 우아한 종료를 적용하여, 스케줄러가 종료할 워커에게 짧은 시간 여유(예: 30초이지만 보통 몇 초면 충분)를 준다. `scale_in()` 요청을 받으면, 리더는 새 통신 토폴로지를 구성하고 남은 워커에게 전달한다. 리더는 또한 미래 타임스탬프를 모든 워커에게 보내며, 해당 시점에 종료 워커는 떠나고 남은 워커는 새 통신 토폴로지로 전환해야 한다. 우아한 종료를 사용하면, 종료 워커만 떠나면 되고 남은 워커는 중단하고 기다릴 필요가 없으므로 `scale_in()`의 오버헤드는 무시할 수 있다.

장애 복구. 워커 장애를 포함한 프로세스 장애를 스케일 인의 특수한 경우로 간주한다. EDL은 두 가지 장애 복구 프로토콜을 제공한다: 일관적 복구(consistent recovery)와 근사적 복구(approximate recovery). 일관적 복구는 리더가 주기적으로(예: 1000 미니배치마다 또는 10분마다) 영구 저장소에 체크포인트를 작성하도록 요구한다. 근사적 복구는 살아남은 워커들로 새 통신 토폴로지를 구성하고 현재 미니배치를 재실행할 수 있다. 기본적으로 EDL은 일관적 복구를 사용한다.

4.3 동적 데이터 파이프라인

기존 DL 시스템은 훈련 시작 전에 데이터셋을 워커 간에 분할하고, 각 워커는 매 에포크에서 자신에게 할당된 파티션을 처리한다. 이 정적 데이터 할당 방법은 실제로 잘 작동하지만, 탄력적 DL에서는 유연성이 부족하고 데이터 관리가 복잡해진다.

탄력성을 지원하기 위해, EDL은 온디맨드 방식으로 워커에게 데이터 파티션을 동적으로 할당한다. 데이터셋은 논리적으로 d개 파티션으로 분할되며, d는 워커 수보다 충분히 크면서도 파티션 크기가 고대역폭 데이터 읽기를 허용할 만큼 충분히 크다. 분할은 메타데이터 수준에서만 수행되며(파일 이름과 오프셋 기록), 데이터셋은 물리적으로 분할되지 않는다.

워커가 새 파티션을 필요로 하면, `elastic_shard_generator()`가 반환한 제너레이터 객체의 `next()` 메서드를 호출하여 리더에게 데이터 읽기 요청을 보낸다. 리더는 다음 미할당 파티션의 메타데이터(예: 파일 경로, 오프셋, 길이)로 응답한다. 워커는 분산 파일 시스템(예: HDFS)에 비동기 I/O 요청을 발행하여 해당 파티션을 읽는다.

진행 상황 추적을 위해 각 워커는 현재 파티션의 오프셋을 기록한다. 새 워커가 작업에 참여하면, 리더는 단순히 미처리(또는 부분 처리된) 파티션을 할당한다. 워커가 우아한 종료로 떠날 때는 현재 파티션의 메타데이터와 오프셋을 리더에게 보고하여, 리더가 남은 미처리 데이터를 다른 워커에 할당할 수 있게 한다.

4.4 구현 세부사항

Horovod v0.16.1을 수정하여 Boost.asio를 사용한 플러그인으로 EDL 데몬을 구현했으며, 약 4,000줄의 코드로 이루어져 있다. TCP를 사용하여 리더와 클러스터 관리자 간에 연결한다. 메시지 지연을 줄이기 위해 TCP 소켓에서 Nagle 알고리즘을 비활성화했으며, 한 메시지 전송의 평균 지연은 56μs이다.

파일 시스템으로부터의 훈련 데이터 읽기 지연을 숨기기 위해, 각 워커는 생산자-소비자 데이터 파이프라인을 운영한다. CPU와 GPU 사이에 핑퐁 버퍼(이중 버퍼)가 유지된다. GPU 계산과 호스트-디바이스 데이터 이동을 겹쳐, 여러 미니배치의 훈련 샘플을 주 메모리에서 GPU로 프리페칭한다.

5. 탄력성의 활용

이 섹션에서는 EDL이 DL 클러스터 스케줄링에 이점을 가져오고, 낙오자 처리, 성능 프로파일링, 워커 마이그레이션과 같은 중요한 시스템 기능을 구현하는 데 사용될 수 있음을 논의한다.

5.1 탄력성 인지 DL 스케줄링

EDL을 사용하여 (1) 처리량과 GPU 효율성 간의 트레이드오프를 조정하고, (2) 클러스터 부하 변동에 적응하여 클러스터 활용률과 JCT를 개선하며, (3) 일시적 유휴 자원을 잘 활용할 수 있다.

새로운 스케줄러를 개발하는 것은 본 논문의 범위를 벗어나므로, 최단 작업 우선(shortest-job-first) 원칙에 기반한 최신 GPU 클러스터 스케줄러인 Tiresias를 확장한다. Tiresias는 여러 그룹 G_0, G_1, ...에서 작업을 관리하며, 인덱스가 작은 그룹이 더 높은 우선순위를 가진다.

Tiresias에 탄력성 인지 스케줄링을 가능하게 하기 위해, 알고리즘 1로 스케줄링 프로토콜을 확장한다. 대기 작업 수가 임계값 N을 초과하면 **압축(Compaction)**이 트리거되어(1-6행), 실행 중인 작업에서 GPU를 scale-in으로 제거하고 대기 작업에 할당한다. 대기 작업이 없지만 유휴 GPU가 있으면 **확장(Expansion)**이 트리거되어(7-13행), GPU를 할당했을 때 가장 큰 이득을 얻는 실행 중인 작업에서 유휴 GPU를 선택한다.

이 새로운 스케줄링 알고리즘을 Elastic-Tiresias라 부른다. 직관적으로, Elastic-Tiresias의 두 규칙은 클러스터 부하가 높을 때 GPU 효율성을 향상시키고, 부하가 낮을 때 유휴 자원을 최대한 활용하는 것을 목표로 한다.

5.2 EDL의 추가 활용 사례

낙오자 완화. 높은 GPU 온도(클럭 주파수 저하 유발)와 동일 위치 작업의 강한 간섭 등의 이유로 일부 워커가 낙오자가 될 수 있다. EDL은 과거 여러 미니배치의 그래디언트 동기화 요청 통계를 기반으로 낙오자를 탐지한다. 워커가 지속적으로 다른 워커보다 느리면(예: 미니배치당 시간이 10 미니배치의 중앙값의 1.2배 이상), 리더는 `scale_in()` 연산을 트리거하여 무시할 수 있는 오버헤드로 해당 워커를 제거할 수 있다.

성능 프로파일링. 다양한 병렬도와 배치 계획에서 DNN 훈련 작업의 성능에 대한 분석 모델을 구축하는 것은 중요하지만 일반적으로 어렵다. EDL의 `profile()` 메서드를 사용하여 (min, max) 범위의 병렬도에서 런타임 성능을 쉽게 측정할 수 있다. `scale_in()`이 `scale_out()`보다 오버헤드가 훨씬 적으므로, EDL은 최대 병렬도에서 프로파일링 작업을 시작하고 최소 병렬도까지 점진적으로 스케일 다운한다.

워커 마이그레이션. 때때로 스케줄러는 작업의 워커를 한 머신에서 다른 머신으로 이동시켜야 한다(예: 워커를 동일 위치에 배치하여 통신 비용을 줄이기 위해). EDL에서 워커 마이그레이션은 먼저 스케일 인으로 대상 머신의 워커를 제거한 후, 스케일 아웃으로 목표 머신에 새 워커를 추가하여 쉽게 수행할 수 있다. 통신 토폴로지가 한 번만 전환되는 단일 마이그레이션 연산으로 이 절차를 더 최적화할 수 있다.

6. 실험 결과

8대의 머신으로 구성된 클러스터에서 EDL을 평가했다. 각 머신은 96코어 Intel CPU, 8개의 NVIDIA Tesla V100 SMX2 GPU, 256GB RAM을 갖추고 있다. 머신들은 100Gbps InfiniBand로 연결되어 있다. 4개의 인기 DNN 모델을 사용했으며, 모델 크기와 배치 크기는 표 2에 보고되어 있다.

표 2. 모델 통계

	Inception3	ResNet50	VGG16	Transformer
모델 크기(MB)	92	98	528	24
배치 크기(GPU당)	64	64	32	2,048

기본적으로 강한 스케일링을 사용했다. 합산 배치 크기는 GPU당 배치 크기 × 초기 GPU 수로 고정하고 스케일링과 무관하게 변경되지 않았다. 수렴 시간과 모델 품질(예: 분류 정확도)은 테스트하지 않았는데, 이는 강한 스케일링과 동기식 훈련에서 병렬도 조정이 모델의 품질에 영향을 미치지 않기 때문이다. 실험의 주요 결과를 다음과 같이 요약한다.

- EDL은 스케일링이 없을 때 정상 훈련에 무시할 수 있는 오버헤드를 도입하며, Stop-Resume에 비해 병렬도 조정의 오버헤드를 크게 줄인다.
- EDL이 가능하게 하는 효율적인 탄력성은 프로파일링, 낙오자 완화, 작업 마이그레이션과 같은 스케줄링 프리미티브를 효율적으로 지원한다.
- 탄력성을 활용하여, 섹션 5.1에서 소개한 Elastic-Tiresias 알고리즘은 원래 Tiresias에 비해 더 높은 GPU 활용률을 달성하고 JCT를 크게 단축한다.

6.1 탄력성의 오버헤드

정적 병렬도에서의 성능. DNN 훈련 작업이 대부분의 시간 동안 정적 병렬도(즉, 전체 실행 동안 GPU 수를 고정)로 실행되므로, EDL의 탄력성 설계(예: RPC 기반 조정, 동적 데이터 파이프라인)가 정상 훈련에 거의 오버헤드를 부과하지 않는 것이 중요하다. 최신 분산 DL 프레임워크인 Horovod와 비교하여 이를 검증했다. 최대 32개 GPU를 사용하여 다양한 DNN 모델의 처리량(500개 이상의 미니배치 평균)을 측정했다. EDL은 Horovod와 비교 가능한 처리량을 달성하며, 차이는 최대 3%에 불과했다.

스케일링 오버헤드. 스케일 인에서 EDL은 훈련을 중단하지 않고 우아한 종료를 사용하여 종료 워커를 제거한다. 스케일 아웃에서 EDL은 최신 모델을 새 워커에게 전달하기 위해 짧은 시간 동안 훈련을 중단해야 한다. 비교적으로, Stop-Resume은 스케일 인과 스케일 아웃 모두에서 전체 병렬도 조정 기간 동안 모든 워커를 중단해야 한다.

EDL의 스케일 아웃 중단 시간(20회 시행 평균)을 보고한다. 모든 경우에서 EDL의 중단 시간은 Stop-Resume보다 한 자릿수 이상 짧다. EDL의 경우 ResNet50의 중단 시간이 긴 이유는 복잡한 모델 아키텍처와

많은 작은 텐서를 가지고 있어, 작은 텐서 전송 시 통신 대역폭을 효율적으로 활용하기 어렵기 때문이다. 모델 크기가 네트워크 대역폭에 비해 작으므로, 모든 경우에서 EDL의 중단 시간은 1초 미만이다.

스케일 아웃 시 EDL과 Stop-Resume의 자원 손실(GPU × 시간)을 비교한다. EDL의 자원 손실이 Stop-Resume보다 현저히 작다. 이는 Stop-Resume이 전체 스케일링 연산 동안 모든 GPU를 중단시키는 반면, EDL의 경우 새로 추가된 GPU만 스케일링 중에 사용되지 않고, 기존 GPU는 모델 전달을 위해 짧은 시간만 중단되면 되기 때문이다.

6.2 EDL 사용의 이점

성능 프로파일링. EDL과 Stop-Resume의 프로파일링 작업(2-8 GPU에서 훈련 성능을 10 미니배치 동안 테스트) 소요 시간을 보고한다. EDL이 Stop-Resume의 약 10%의 시간만으로 동일한 프로파일링 작업을 수행했다. 이는 Stop-Resume이 매 병렬도마다 고가의 컨텍스트 초기화 비용을 반복적으로 지불하는 반면, EDL은 처음에 한 번만 컨텍스트 초기화 비용을 지불하고 그 후 저오버헤드 스케일 인 연산을 사용하기 때문이다.

낙오자 완화. 16개 GPU로 실행되는 작업에 대해 미니배치당 시간의 1/3만큼 그래디언트 동기화 요청을 지연시켜(계산 능력을 최대의 75%로 제한) 수동으로 낙오자를 생성했다. 전체 처리량도 정상 경우의 약 75%로 저하된다. EDL을 과거 10 미니배치의 통계를 기반으로 낙오자를 탐지하도록 구성했다. 테스트한 모든 작업에서 EDL은 10초 이내에 낙오자를 탐지하고 5초 이내에 scale-in을 사용하여 제거했다. 낙오자 제거 후 훈련 처리량은 정상 경우의 약 94%(GPU 1개 적은, 즉 제거된 낙오자)로 회복되었다.

워커 마이그레이션. 대형 모델의 기계 간 통신 비용이 높으므로 작업의 GPU를 동일 위치에 배치하는 것이 중요하다. 2대의 머신에서 실행되는 작업(각 4개 GPU 사용)을 고려하여, EDL을 사용하여 한 머신으로 마이그레이션하여 8개 GPU로 실행했다. 대형 모델(예: VGG16)의 경우 마이그레이션 후 처리량이 약 50% 증가했다. 마이그레이션 비용은 스케일 아웃과 유사하며 대상 머신의 훈련은 1초 미만만 중단되었다.

일시적 자원 활용. EDL이 일시적 유휴 GPU 자원으로부터 이점을 가져올 수 있는지 검증하기 위해, 4개의 영구 GPU로 ResNet50을 훈련하고 같은 머신에 1, 2, 4개의 유휴 GPU를 4분마다 추가하는 실험을 수행했다. 네 가지 방식을 사용했다: (1) 유휴 GPU를 사용하지 않는 Baseline, (2) 스케일링에 Stop-Resume을 사용하는 Stop-Resume, (3) 스케일링에 EDL을 사용하는 EDL, (4) 스케일링이 즉시 완료된다고 가정하는 이상적인 Ideal. EDL이 Ideal 처리량의 최소 97%를 달성한 반면, Stop-Resume은 높은 스케일링 오버헤드로 인해 Baseline보다도 낮은 성능을 보였다. Stop-Resume이 유휴 GPU 1개로 Baseline을 능가하려면 최소 11.7분의 일시적 간격이 필요한 반면, EDL은 워커 시작 시간보다만 길면 된다.

6.3 클러스터 스케줄링 성능

합성 워크로드. 스케줄링에서 EDL 사용의 이점을 보이기 위해, 합성 워크로드를 생성했다. 4대의 머신(각 8개 GPU)에 30초마다 작업을 제출하여, 16개 작업이 제출될 때까지 계속했다. 각 작업은 TensorFlow 공식 벤치마크의 9개 인기 DNN 중 무작위로 선택된 모델을 훈련했으며, 모든 작업은 기본적으로 4개 GPU를 사용했다.

Static과 Elastic(즉, EDL 사용) 두 가지 스케줄링 전략을 비교했다. Static은 처음 8개 작업까지 정적 병렬도 4로 모든 GPU를 점유하고, 이후 새 작업은 대기 큐에 넣었다. Elastic은 작업을 가장 적게 부하가 걸린 머신에 할당하고 유휴 GPU를 활용하여 스케일 아웃했다. 결과, Elastic이 거의 항상 Static보다 높은 클러스터 처리량을 달성했다.

프로덕션 클러스터 시뮬레이션. 대규모 GPU 클러스터에서 EDL 사용의 이점을 보이기 위해, Elastic-Tiresias와 Tiresias를 비교했다. Microsoft 프로덕션 클러스터의 트레이스 데이터(100,000개 이상의 훈련 작업 포함)를 기반으로 시뮬레이션을 수행했다.

표 3. 작업 완료 시간(JCT) 통계 (초)

	Tiresias	Elastic-Tiresias	감소율(%)
평균	235,068	24,658	89.5%
중앙값	1,080	561	48.1%
95번째 백분위수	1,914,470	88,886	95.4%

탄력성이 활성화되면 Tiresias의 JCT가 크게 감소했다. Elastic-Tiresias가 Tiresias보다 더 높은 GPU 활용률과 클러스터 효율성을 달성했다. Elastic-Tiresias의 GPU 활용률이 더 높은 이유는 유휴 GPU를 활용하기 위해 작업을 스케일 아웃하기 때문이다. 클러스터 효율성 곡선은 GPU 활용률 곡선과 높은 상관관계를 보이며, 유휴 GPU 활용이 더 높은 클러스터 효율성으로 이어지고, 이는 다시 JCT 개선으로 이어진다.

7. 관련 연구

딥러닝 스케줄러. YARN, Mesos, Omega, Borg와 같은 전통적인 클러스터 관리자 대신, 최근 Optimus, Gandiva, Tiresias 등 다수의 DL 특화 스케줄러가 멀티테넌트 GPU 클러스터를 위해 제안되었다. Optimus는 Stop-Resume 방식을 사용하여 주기적으로 MXNet의 파라미터 서버/워커 수를 조정하여 JCT를 최소화한다. Gandiva는 런타임 측정에 따라 마이그레이션, 확대-축소(grow-shrink), 프로파일링, 일시 중지-재개(suspend-resume) 등 다양한 메커니즘을 도입하여 자원 할당을 조정한다. EDL은 낮은 오버헤드의 탄력성을 제공하는 시스템으로 자리매김하며, 더 빈번한 병렬도 조정과 마이그레이션 및 프로파일링 같은 스케줄링 메커니즘을 효율적으로 지원함으로써 기존 GPU 스케줄러와 협력할 수 있다.

탄력적 ML/DL 시스템. 머신러닝 시스템은 보통 파라미터 서버 아키텍처에 기반하며, 로지스틱 회귀(LR)나 잠재 디리클레 할당(LDA) 같은 분산 ML 워크로드를 처리한다. Baidu의 Paddle EDL은 파라미터 서버 아키텍처에 기반한 DL 시스템으로 Kubernetes와 통합되어 있다. DL2는 (파라미터 서버 기반) MXNet에서 탄력성을 지원하지만, 새 워커 추가의 오버헤드를 어떻게 숨기는지와 훈련 데이터가 동적 워커 집합 간에 어떻게 분할되는지는 불분명하다. 동시 연구인 AutoScaling도 Allreduce 아키텍처에서 탄력성을 지원하고 스케일링 오버헤드를 줄이려 한다. 그러나 AutoScaling은 멀티테넌트 클러스터 관리 개선이 아닌 단일 작업의 비용과 효율성 간 트레이드오프에 초점을 맞춘다.

일시적 자원을 위한 시스템. 클라우드에서 온디맨드 인스턴스보다 훨씬 저렴한 선점형 인스턴스의 가격으로 인해, 많은 시스템이 일시적 자원을 활용하도록 설계되었다. Proteus는 신뢰할 수 있는 서버 노드에서 모델을 관리하고 워커를 동적으로 추가/제거하는 파라미터 서버 기반 ML 시스템이다. Hourglass는 그래프를 마이크로 파티션으로 분할하고 자원 변경 시 머신 간에 재할당하는 그래프 처리 시스템이다. 일시적 워커는 보통 수시간 지속되지만, EDL은 일시적 GPU 자원이 수분 동안만 사용 가능한 더 엄격한 상황을 고려하며, 이는 낮은 오버헤드의 탄력성을 필요로 한다.

8. 결론

우리는 낮은 오버헤드로 탄력적 GPU 활용을 지원하는 EDL을 제시했다. EDL은 부하 변동 적응을 통한 자원 활용률 향상, 일시적 유휴 GPU 활용 극대화, 성능 프로파일링, 낙오자 완화, 작업 마이그레이션 등 여러 방면에서 멀티테넌트 GPU 클러스터 관리에 이점을 가져올 수 있다. 실험을 통해 이러한 응용에서 EDL을 사용하여 상당한 성능 이점을 얻을 수 있음을 보였다.

감사의 글

논문의 품질을 크게 향상시키는 건설적인 의견을 제공해 주신 심사위원분들께 감사드린다. 이 연구는 HKSAR의 RGC로부터 GRF 14208318의 지원을 받았다.